॥ श्रीगणेशाय नमः ॥

Data Warehousing

A data warehouse is a large centralized repository of data collected from heterogeneous sources.
- Designed to support business intelligence (BI), specially analytics and reporting.



Regular DBMS

← To perform day-to-day business functionalities e.g. performing transactions, shipping related info, populating lists of products, etc.
- CRUD operations
- expected to be up, online, frequent, and concurrent.

- focus on historical data
- predictive views of business operations instead of individual data.
- e.g. customers who spent the most in a month



Data warehouse

A data warehouse is a data store that is
- subject-oriented (organized around major subjects)
- integrated
- time-variant
- non-volatile

OLTP: online transaction processing - major tasks of operational DBs
OLAP: online analytics processing - major tasks of data warehouses

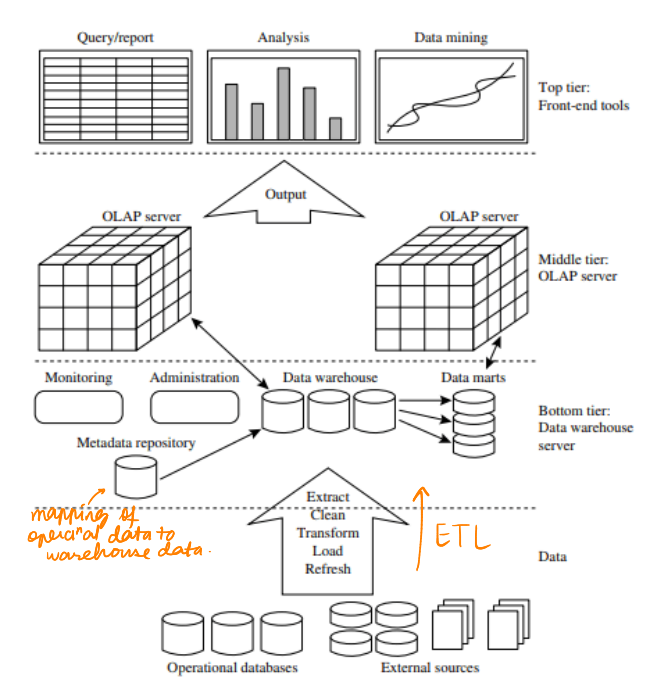
	OLTP	OLAP
Purpose	Manages daily transaction data.	Manages more complex queries and data analysis.
Store	Generally normalized.	Generally un-normalized.
Operations	CRUD	Read-heavy operations and aggregations.
Volatility	Highly volatile, frequent updates.	Less or non-volatile.
Data volume	Lesser	Larger.
Response time	Faster (to support real-time operations).	Slower.
Integrity	Data integrity and consistency is maintained.	More concerned on data accuracy and less upon real-time consistency.
Backups	Required frequently.	Less frequent.
DB design	ER data model and application-oriented DB design.	Adopt either star model or snowflake model and a subject-oriented design.

Why 2 different data stores?
- We want high performance on both systems - transactional (daily use) and analytics (less frequent).
- OLAP queries are complex and time consuming, so can degrade the performance of potential tasks.
- Concurrency control and recovery mechanisms applied to OLAP can result in delaying the execution, thus decreasing the throughput.

Data warehouse architecture

Architecture is the overall design and structure of a warehouse system. It defines how data is collected, integrated, transformed, stored, and accessed for a decision-making process.
The main aim is to ensure efficiency and scalability.

3-tier architecture:



Front-end client layers
- Tools for querying, reporting, visualizations, analysis, and predictions.

OLAP servers
- 2 types:
Relational OLAPs (ROLAPs):
operations on multi-dim data are mapped to standard relational operations.
Multi-dimensional OLAPs (MOLAPs):
special sources that perform operations directly on multi-dim data.

Extraction layer
- It has various data sources, from where data is extracted.
- ETL tools are used in the bottom tier.

ETL process:



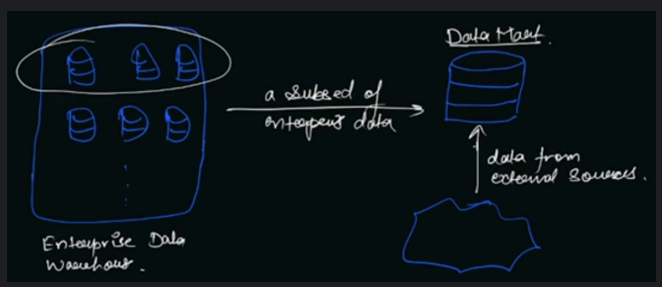
E
Extract:
- Gathering data from multiple sources.
- Extraction wrappers: scripts/ procedures that extracts regular subcontent of an unstructured information and translate it to relational form.

T
Clean: rectifying errors.
Transform:
- Converting the format of raw data to the format that the warehouse is designed for.

L
Load:
- Sorting, summarizing, consolidating, and aggregating the data.
Refresh:
- Propogating updates from data sources to warehouses.
- 2 ways for refresh: the long haul and the last mile (nice...)
  long haul: takes time, can be a full load or a batch-processing one. E.g. update the warehouse at the end of the year.
  last mile: lightweight, quicker, incremental in nature; E.g. updating daily transactions.

Enterprise data warehouses

- Collection of all information spanning the entire organization/enterprise.
- Large scaled and centralized repository.
- Provides corporate-wide data integration and its cross-functional scope.
- Takes years to design and build these warehouses.

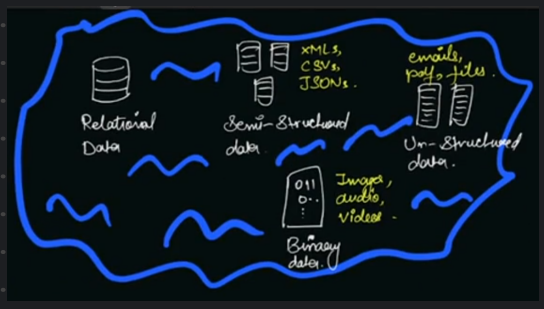


Data marts

- A subset of the corporate-wide data.
- Provide value to a specific group of people, such as within a department.
- More confined scope (subject-oriented).
- Implementation cycle is lesser (say a few weeks or so).
- 2 types:
  a. Dependent data marts: data is sourced directly from data enterprises;
  b. Independent data marts: data is sourced from many operational systems, external sources (3rd party integrations);

Data lakes

A data lake is a centralized repository that allows us to store all kinds of data: structured, semi-structured, and unstructured, at any scale.

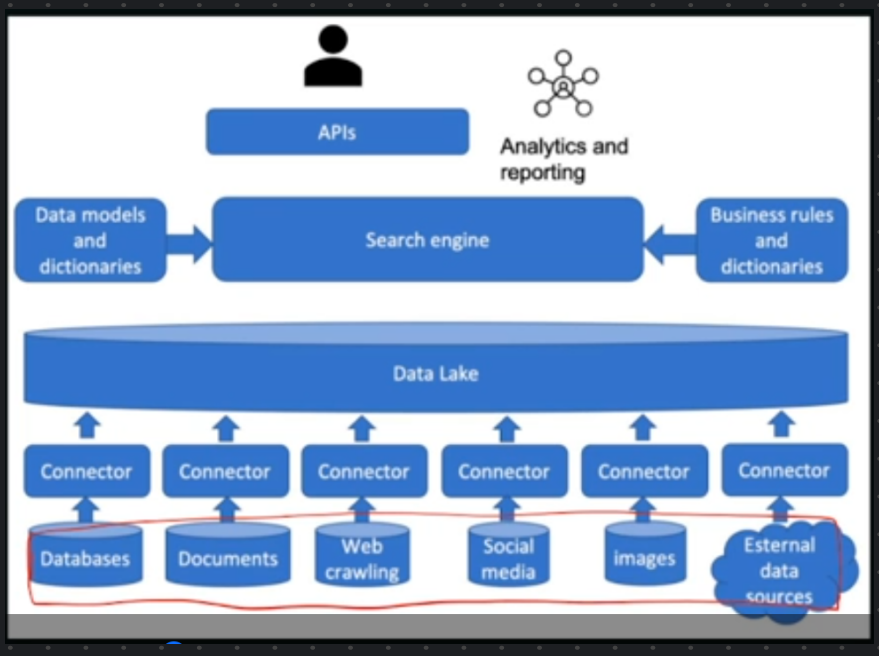


Why data lakes?

- Building a warehouse is time-consuming. We need to understand the business process, analyze data sources, and design the models.
- More scalable than warehouses (upto petabytes).
- More flexible: data warehouses have a well-defined structure, thus to address new queries or business changes, it'll take time to upgrade.
  Lakes store raw data, thus are more flexible to the upcoming changes.

Data lake storage layers

Data lake architecture



→ to make the searching more business oriented

Challenges:

- Difficult searching and querying.
- Data quality is not guaranteed.

Dare warehouse modelling

- It is the process of designing the architecture of a data warehouse.
- Organizing data into schema and ensuring that it is structured, which will ensure efficient querying and analysis.

Data cube

- It allows data to be modelled in multiple dimensions.
- A multi-dim array of values.

Some key terms:

- Subject: The theme of the multi-dim data model.
- Dimension: Possible perspectives or entities that the subject needs to be analyzed. These perspectives are modeled as dimensions.
- Measurements (facts): Numeric values that need to be analyzed. Mostly they are aggregations.

Example
Say the subject is 'sales' (of some organization maybe).
Dimensions could be: time, items, branch, location, etc.
Facts: units_sold, location_expenses, amount_sold, etc.

NOTE:
- Fact table: that stores the names of facts and keys referencing to the related dimension table.

	location = "Vancouver"			
time (quarter)	item (type)			
	home entertainment	computer	phone	security
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Not dim, it is a fixed loca^n
dimensions

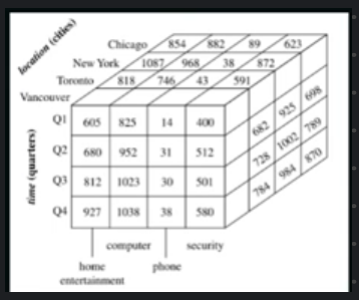
← Dimensions:
- time
- item

← Facts:
- amount_sold

View of sales data with a third dimension {time, item, location}:

time	location = "Chicago" item				location = "New York" item				location = "Toronto" item				location = "Vancouver" item			
	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.	home ent.	comp.	phone	sec.
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Cubical representation:

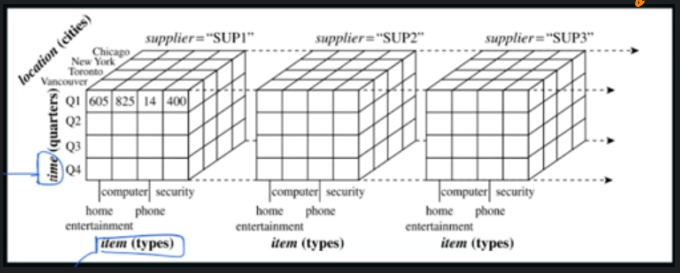


Can we get a 1-D representation for the table for Vancouver?
Yes, we'll just add up for all item types.
(or some other aggregation in some other case)

] that 1D thing is called a cuboid as well.

If we want to add a new dim?

chain of 3D cubes; each one represents a supplier

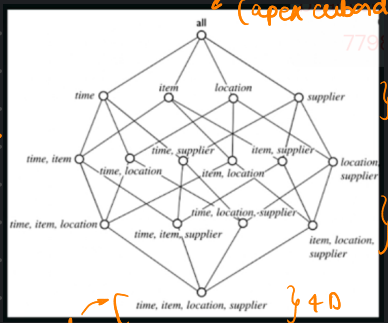


Dims:
- time
- types
- loca^n
- supplier

This where thing is a called a cuboid.

Once these dimensions are fixed, we can form cuboids for each possible subsets of the given dimensions.
These subsets are cuboids as well.
These subsets are together called a lattice of cuboids.
Their collection is called a data cube.

data cube] 0-D (apex cuboids)



The individual nodes are cuboids, the whole thing is a cube.
The structure itself is called a lattice.

n-d base cuboid (lowest level of summarization/aggregation)

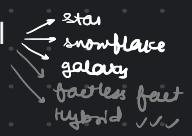
Steps involved in data warehouse modeling:

1. Requirement analysis
   - finding key metrics
   - defining the scope.
2. Concept modeling
   - identifying the entities, dimensions, and facts.
   - forming a high-level model.
3. Logical modeling
   - designing the schema, defining tables (fact and dim)
4. Physical modeling
   - indexing, partitioning, implementing aggregations,
5. Data sourcing and Integration
   - finding the data sources.
   - ETL process
6. Data Loading
   - Incremental data load
   - Data archiving
7. Performance Tuning
   - optimizing queries
   - monitoring and analysis of performance
8. Maintenance and Documentation
   - regular changes with business needs.
   - documentation, model, schema, data flow diagrams, ETL steps, etc.

Schema modeling

- Every data warehouse requires a concise, subject-oriented schema that facilitates scanning a large amount of data efficiently.
- Efficiency, scalability, usability is needed.
- There are many types of data models that are used in warehousing and DBMS:
  Relational model (ER), hierarchical model, network model, object-oriented data models, flat model, etc.

Multi-dimensional data model

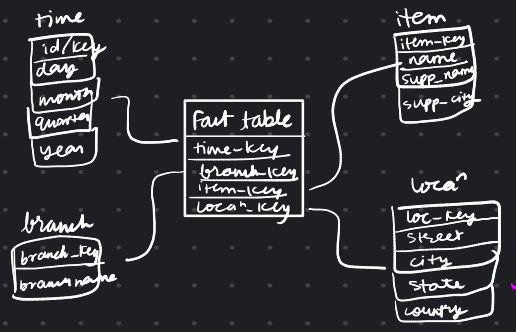


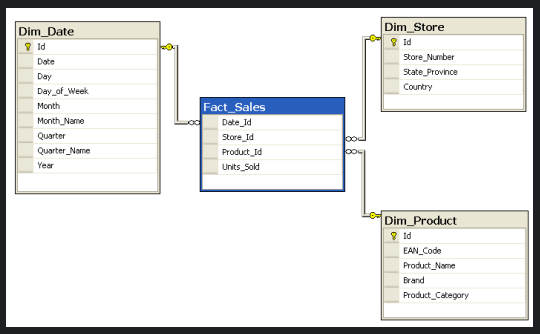
Star schema

- Most commonly used schema as it is simple to implement.
- Contains a large centralized fact table along with a set of smaller attendant tables (dimension tables).

NOTE: There will only be 1 dimension table for a dimension.

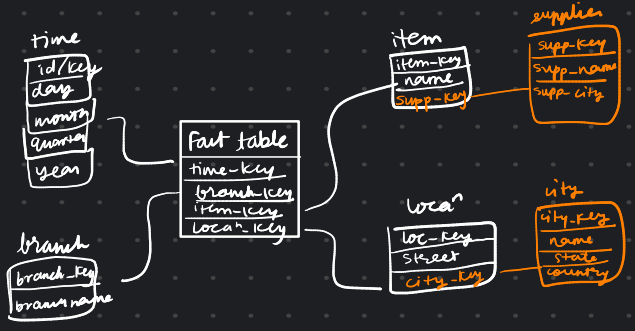
E.g. dimensions: {time, branch, item, location}
facts: {amount_sold, items_sold}

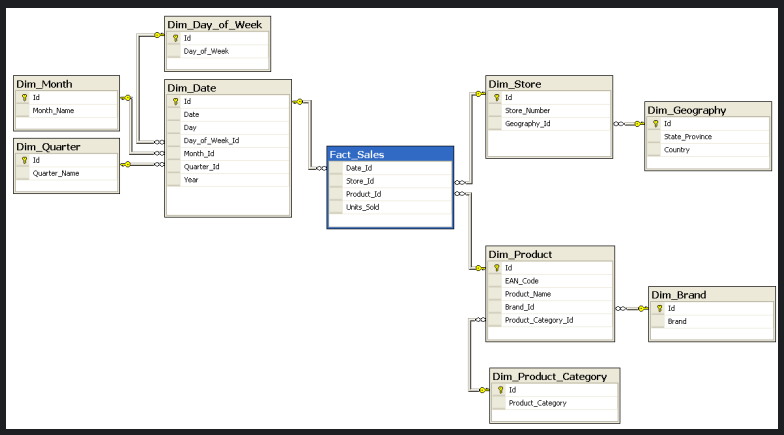




Snowflake schema

- It is similar to the star schema except that we can have multiple tables for a single dimension.
- If we normalize the dimension table in a star schema, we get a snowflake schema.
- The fact table remains the same (still single central factual table).



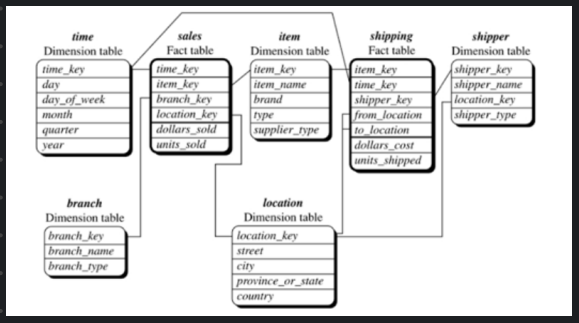


Pros:
- Less storage.
- Easy to maintain.

Con:
- Inefficient read (due to multiple joins and aggregations).

Galaxy schema (fact constellation)

- It has multiple fact tables.
- The dimension tables are shared among these fact tables.



Q. Given the subjects, what schema would we use for modeling?

i. retail sales analysis
dim: {sales_performance, customer_behavior, trend_analysis}

Star. We'll need low latency.
Also, the dims are simple.

ii. financial risk management for a global bank
dim: {risk_aggregation, compliances_and_reporting, scenario_analysis}

Hybrid / galaxy (↑ mostly). The schema seems complicated.

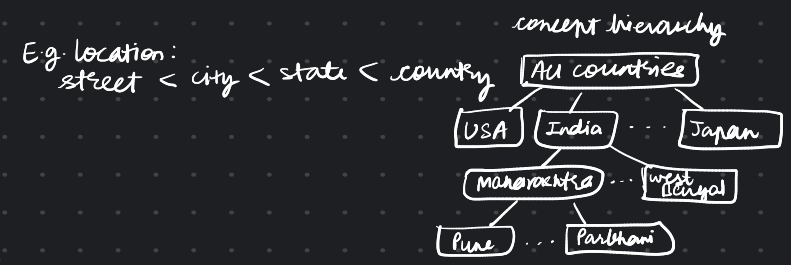
iii. telecom network performance analysis
dim: {network_perf_analysis, resource_optimization, fault mgmt}

Snowflake.
simpler, but there's a sort of network, so hierarchy matters.
data is hierarchical, network can be spread in multiple regions.

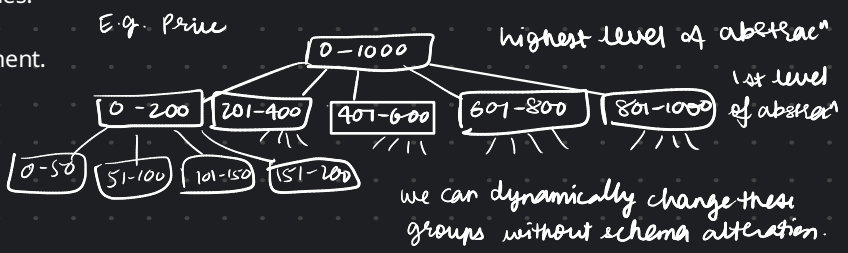
Concept hierarchies

- Organizing data in a hierarchy.
- Sequence of mapping from a set of low-level concepts to high-level ones.
  (low-level → more general; high-level → more specific)

1. Schema-based concept hierarchy:
- Pre-defined hierarchies will be present within the data store schema.
- The structure is rigid and explicitly defined.
- Less flexible, modifications will lead to schema altering.

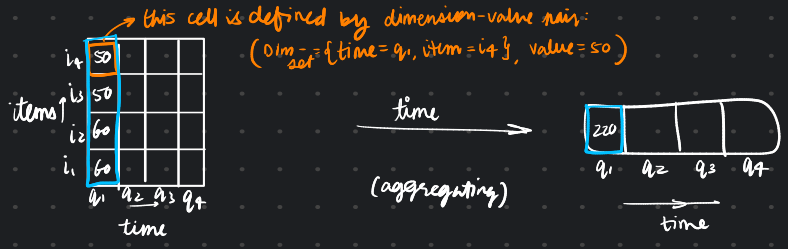


2. Set-group concept hierarchy:
- Hierarchies are formed by logically grouping related data values.
- There's no explicit schema.
- It is more flexible, dynamically created based on the requirement.
- Altering the logic won't impact the schema.



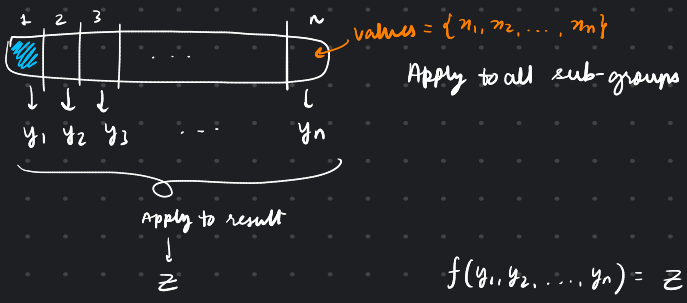
Measures: categorization and computation

A cell in a data cube can be defined by a (set of dimensions, value) pair.
- A measure in a data cube is a numeric function whose value is calculated by aggregating the data corresponding to the dimension-value pairs.



Measures can be organized into categories → Distributive, Algebraic, Holistic } based on aggregation functions

1. Distributive measures:



$$f(y_1, y_2, \ldots, y_n) = z = f(x_1, x_2, \ldots x_n)$$

In these measures, the result of aggregation can be computed by combining the results of aggregations on subsets of data.

E.g. summation is a distributive function.

2, 3, 6, 7, 8, 2, 1, 3 → summation = 32

11 17 4 → summation = 32

Avg is not.

2 2 2 3 6 2 2 2 → AVG = 2.625

4 2.5 4 2 → Avg = 3.125

MAX is distributive.

2 3 6 17 18 2 3 1 → max = 18

6 18 3 → max = 18

2. Algebraic measures:

- The aggregation operation can be expressed as an algebraic combination of other measures.

E.g. AVG is algebraic in nature. It can be written as a combination of SUM and COUNT.

$$AVG() = \frac{SUM()}{COUNT()}$$, VAR, STDDEV(), etc.

3. Holistic measures:

- There is no constant bound on the storage size needed to describe the sub-aggregate.

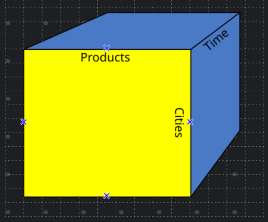
$$AVG() = \frac{SUM()}{COUNT()}$$ ✗ ← NOT holistic since this would have a storage size bound

E.g. MODE, MEDIAN;

OLAP operations

In data warehousing, we also need to have a support for online multi-dimensional analytic queries. These queries would be performing some series of operations (OLAP operations).

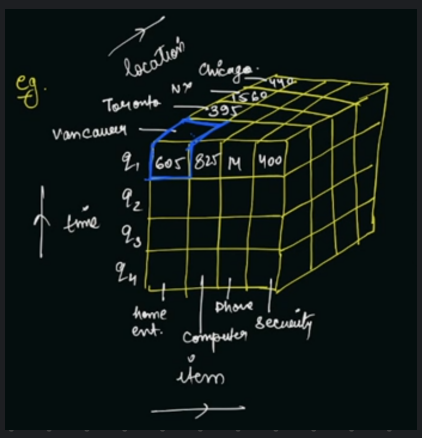
Let us consider a data cube for sales in a company.
3 dimensions : {location, time, and items}

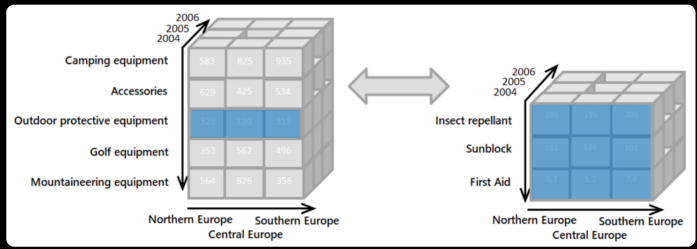


For these operations, consider data cube ≡ data cuboid

1. Roll-up/drill-up

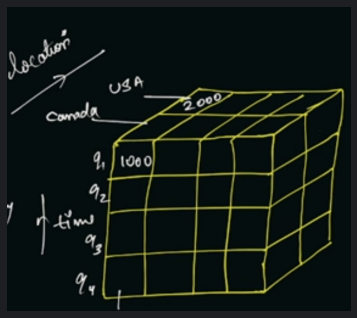
- Perform aggregation on a data cube, either by climbing up a concept hierarchy for a dimension or by dimension reduction.





OLAP drill-up and drill-down

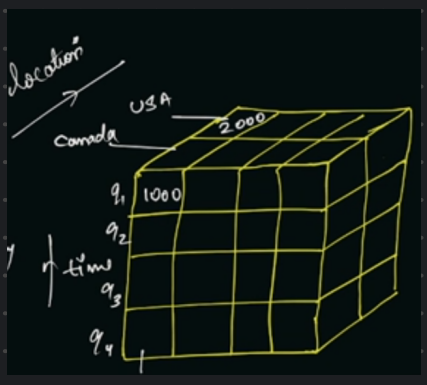
In roll-up, we'd perform an aggregation say city → state. Size of dim. would reduce.
or even state → country.
moving up in the concept hierarchy (location)

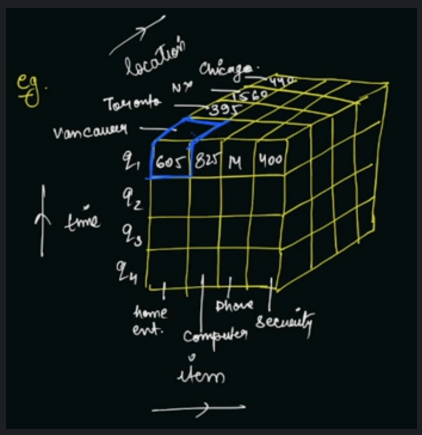


If we go all the way up the concept hierarchy, it's like eliminating/reducing the dimension itself.
street > city > state > country > all
↑
dim reduced

2. Drill-down

- Reverse of drill-up.
- Moving from less detailed → more detailed cubes.
- Either stopping down a concept hierarchy or introducing additional dimensions.



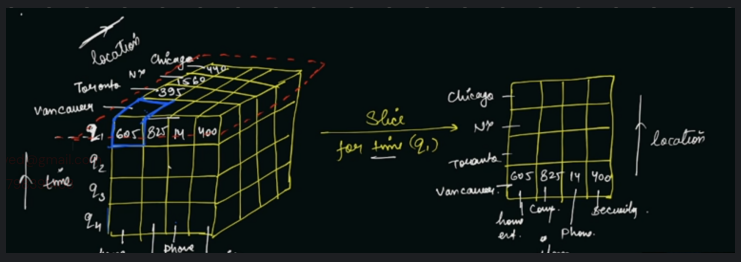


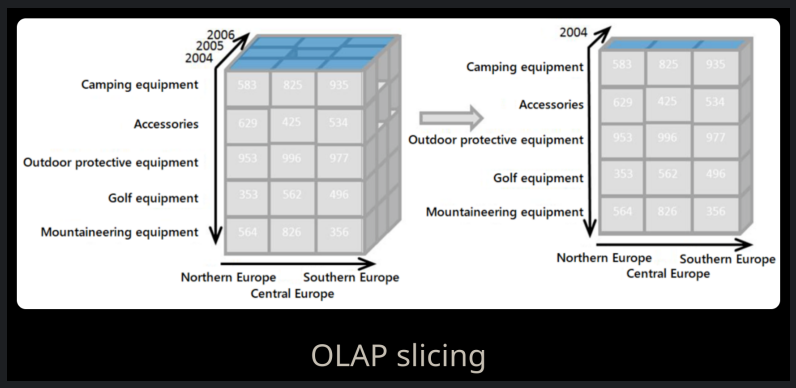
drill-down on months



3. Slice

- Performing a selection on one dimension of a given cube.
- Reducing a data cube to a sub-cube.

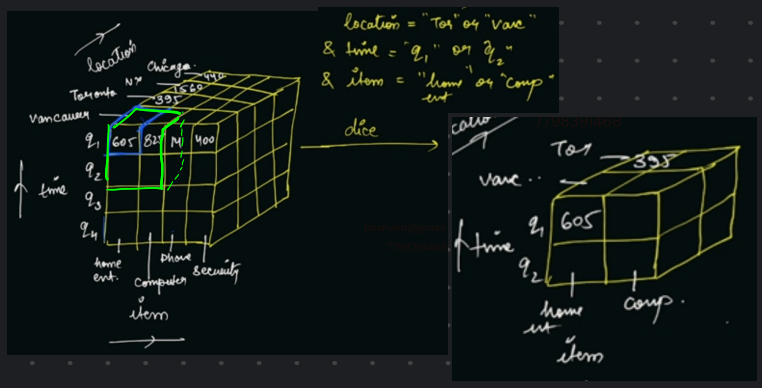


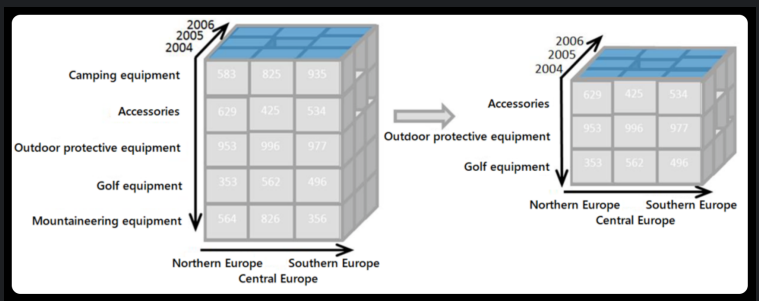


OLAP slicing

4. Dice

- Performing a selection on 2/more dimensions of a given cube.

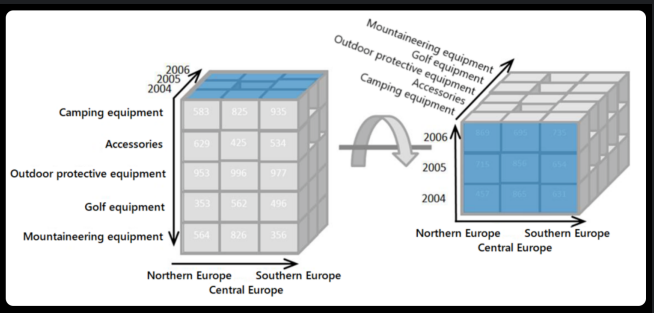




OLAP dicing

5. Pivot

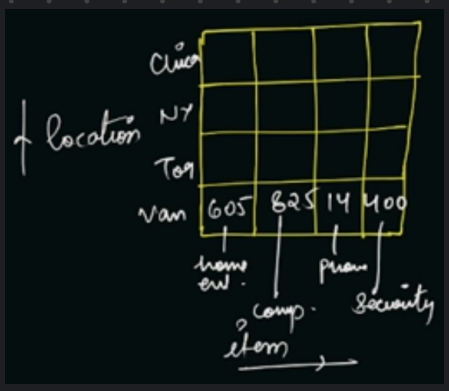
- Visualization operation that rotates the data axes in view to provide an alternative data representation.
- Reorient the cube to provide a different perspective.

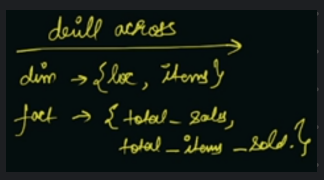


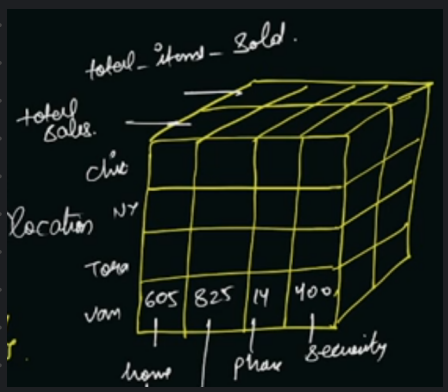
OLAP pivoting

6. Drill-across

- This operation allows us to compare measures from different factual tables that share same dimensions.







7. Drill-through

- It uses relational SQL facilities to drill through the bottom level of a data cube down to its backend relational tables.
- Not applicable for all data → SQL relations might not exist.

